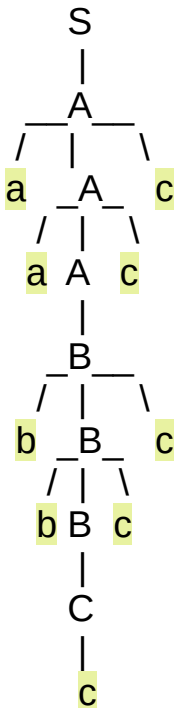
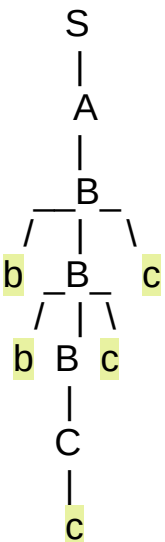
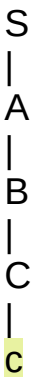
X5 T1. In the task 3 of the previous exercises, there was a pushdown automaton whose language consists of words $\{a^k b^l c^m \mid k+l<m, \quad k,l,m \in N\}$. For example abcccc. The grammar was defined as follows:

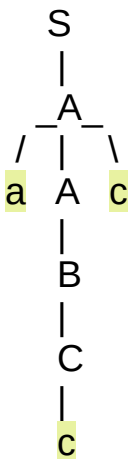
S → A
A → aAc | B
B → bBc | C
C → cC | c

Present parse trees for the words aabbccccc, bbccc, c, acc.

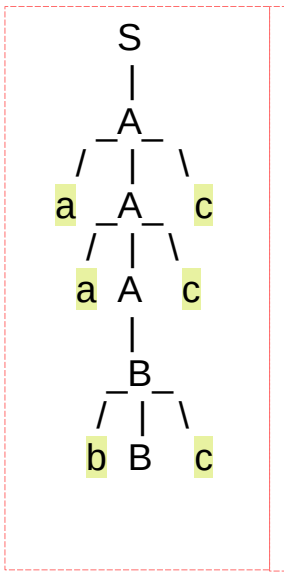


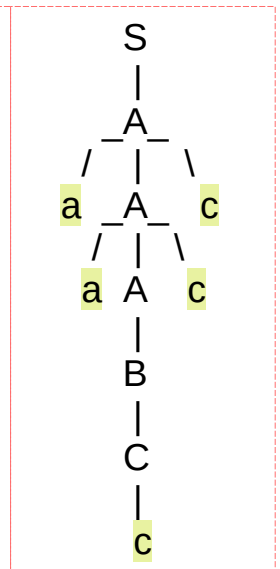






Try to parse aabbccc.





X5 T2. Consider the following context-free grammars over the alphabet Σ = {a, b}. Determine what kind of language each grammar generates. If the language generated by the grammar turns out to be regular, present the corresponding regular expression or finite automaton. If the language is not regular, briefly explain why its recognition exceeds the computational power of a finite automaton.

Examine different possible derivations
S → ε, empty string can be produced
or S → ABS → ABε → aB → aε → a, one a can be produced

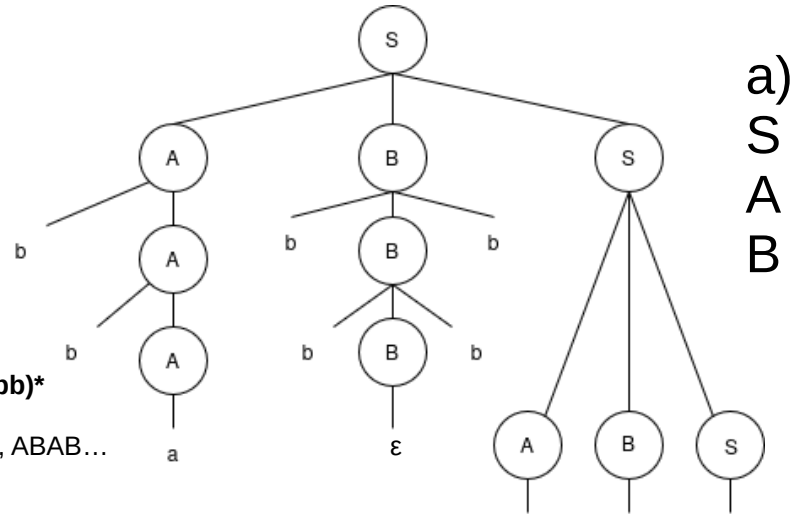
S → ABS → ABε → Aε → A → bA → ba
or S → ABS → ABε → Aε → A → bA → bbA → bba
or S → ABS → ABε → Aε → A → bA → bbA → bbba
Nonterminal A produces 0…n symbols b before the symbol a, that is, **b*a**

S → ABS → ABε → aB → abBb → abεb → abb
or S → ABS → ABε → aB → abBb → abbBbb → abbεbb → abbbb
Nonterminal B produces 0…n strings "bb" after the a produced by A, that is, **b*a(bb)***

Nonterminal S produces a new nonterminal pair AB after an existing AB structure, ABAB…

S → ABS → aBS → aεS → aS → aABS → aaBS → aaεS → aaS → aaε → aa

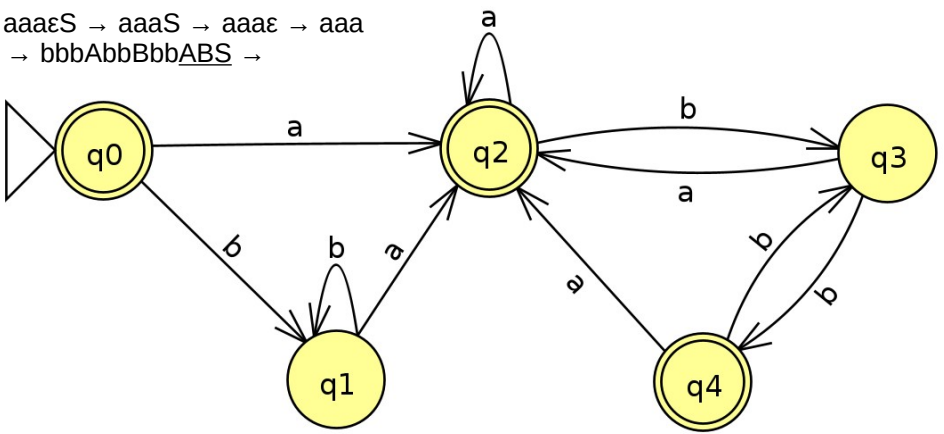
or S → ABS → aBS → aεS → aS → aABS → aaBS → aaεS → aaS → aaABS → aaaBS → aaaεS → aaaS → aaaε → aaa
or S → ABS → bABS → bbABS → bbbABS → bbbAbBbS → bbbAbbBbbS → bbbAbbBbbS → bbbAbbBbbABS →
→ bbbAbbBbbABS → bbbAbbBbbABS → bbbAbbBbbABABS → bbbAbbBbbABABε → …



a)
S → ε | ABS
A → a | bA
B → ε | bBb

The pattern ABABAB... means the string have the form (b*a)(bb)*(b*a)(bb)*(b*a)(bb)*...

(b*a(bb)*)*



X5 T2. Consider the following context-free grammars over the alphabet Σ = {a, b}. Determine what kind of language each grammar generates. If the language generated by the grammar turns out to be regular, present the corresponding regular expression or finite automaton. If the language is not regular, briefly explain why its recognition exceeds the computational power of a finite automaton.

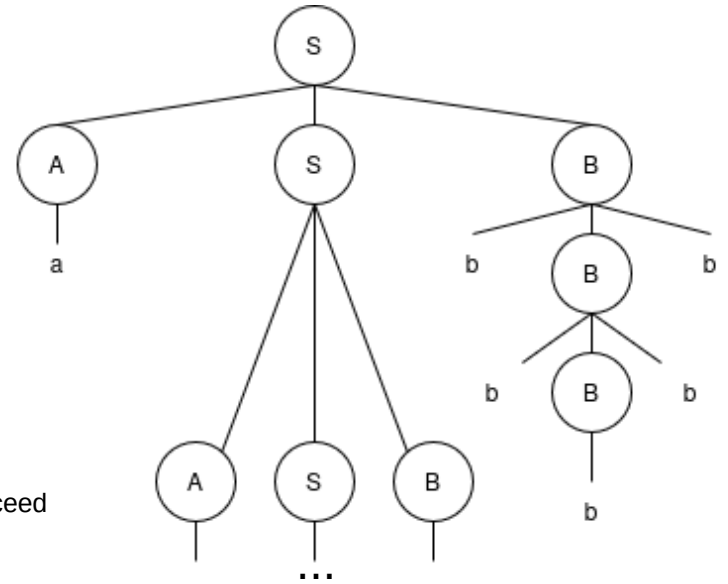
Examine different possible derivations
S → ε, empty string can be produced

Nonterminal A produces one symbol a

S→ ASB→ aεB → aB → abBb → abbb
S → ASB→ aεB → aB → abBb → abbBbb → abbbbb
Nonterminal B produces at least one b, plus additional b's.

S→ ASB→ aεb → ab
S → ASB→ aSb → aASBb → aAεBb → aaεbb → aabb
Under minimal derivations A produces one a, B produces one b, and **nonterminal S** generates strings with equal number of a's and b's.

Because nonterminal B can also produce bBb, total number of b's may exceed the number of a's, but it can never be smaller.



b)
S → ε | ASB
A → a
B → b | bBb

S → ASB→ aSB → aASBB → aaASBBB → aaaεBBB → aaabbb (string $a^n b^n$)
S→ ASB→ aSB → aASBB → aaASBBB → aaaεBBB → aaabBbbBbbBb (each B-nonterminal generates bBb.)
→ aaabbbbbbbbb (string $a^n b^n b^{2n}$)

Therefore there is a dependency: for every a-symbol generated, at least one b must follow in the final string. PDA is required to verify this constraint, because it must ensure that the number of b's is at least as large as the number of a's.

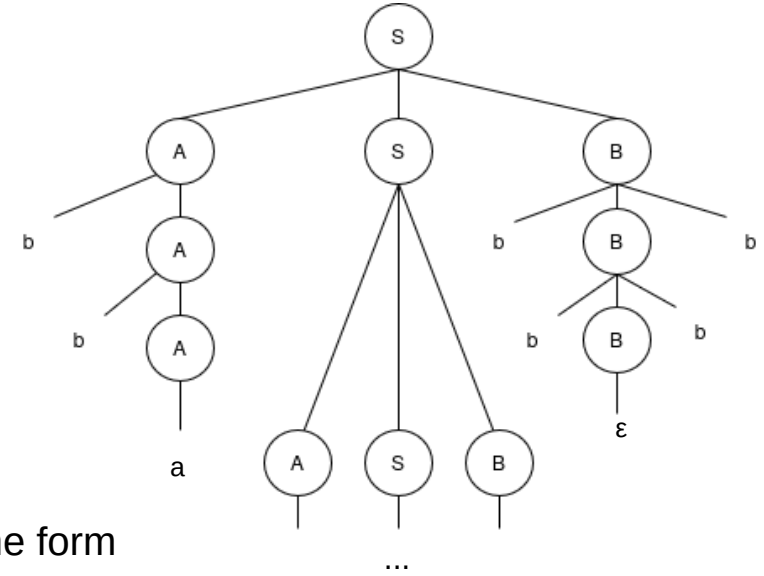
X5 T2. Consider the following context-free grammars over the alphabet Σ = {a, b}. Determine what kind of language each grammar generates. If the language generated by the grammar turns out to be regular, present the corresponding regular expression or finite automaton. If the language is not regular, briefly explain why its recognition exceeds the computational power of a finite automaton.

Examine different possible derivations
S → ε, empty string can be produced

or S → ASB → aSB → aεε → a
or S → ASB → bASB → bbASB → bbbASB …
Nonterminal A produces 0...n symbol b before symbol a

S → ASB → AsbBb → AsbbBbb → …
Nonterminal B produces repetition (bb)*

S → ASB → bASB → bbASB → bbAASBB →
Nonterminal S produces more AB between existing AB



c)
S → ε | ASB
A → a | bA
B → ε | bBb

The pattern AAA...BBB means the string have the form (b*a)(b*a)(b*a)…(bb)*(bb)*(bb)*

A^kB^k where each nonterminal A=b*a and each nonterminal B=(bb)*. The bb's cannot exist without at least one matching a. The grammar forces exactly equal numbers of A's and B's, B (even ε) requires exactly k preceding A's. Pure b-strings like bbbb are impossible since that would need k=0 A's but k>0 B's. The grammar requires counting an unbounded number of A's (each ending in a) to ensure exactly that many B's follow later

X5 T3. Below are context-free grammars of the alphabet Σ = {0,1}. Match the grammars with the following verbal descriptions of languages. One description doesn't have a corresponding grammar. Write the missing grammar.

a) strings that contain the substring 111
b) strings that have an equal number of zeros and ones
c) strings whose length is odd
d) strings that have at least two symbols, and whose first and last symbol are the same

1)
S → BA
A → BBA | ε
B → 0 | 1

2)
S → A111A
A → 0A | 1A | ε

3)
S → 0A0 | 1A1
A → 0A | 1A | ε

4)
S → SS | 0S1 | 1S0 | ε

X5 T4. Consider the following languages over the alphabet Σ={0,1}. Describe in your own words what kinds of strings belong to each language. State what kind of computational model is required to recognize each and why.

a) $\{ 0^n1^n \mid n \geq 1\}$

Binary strings consisting of n zeros followed by n ones. They can be recognized by PDA, that pushes the zeros onto the stack, and when reading ones it pops a zero for each one, ensuring that the two quantities are equal.

b) { (0|1)*10 }

Binary strings that end with the symbols 10, the regular expression is (0|1)*10. They can be recognized by FA, since an automaton that checks just the last two symbols can be implemented with a finite number of states (three states).

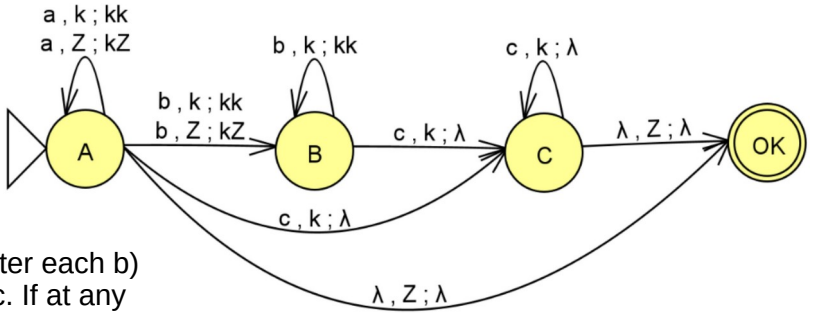
c) $\{ 0^n0^n \mid n \in N \}$

Strings consisting of an even number of zeros, the regular expression is (00)*. The number n does not link two separate blocks to each other. They can be recognized by FA, because with a finite number of states (two states) we can track whether the number of zeros read so far is even or odd.

d) $\{ 0^p1^r0^{p-r} \mid p \geq r \}$

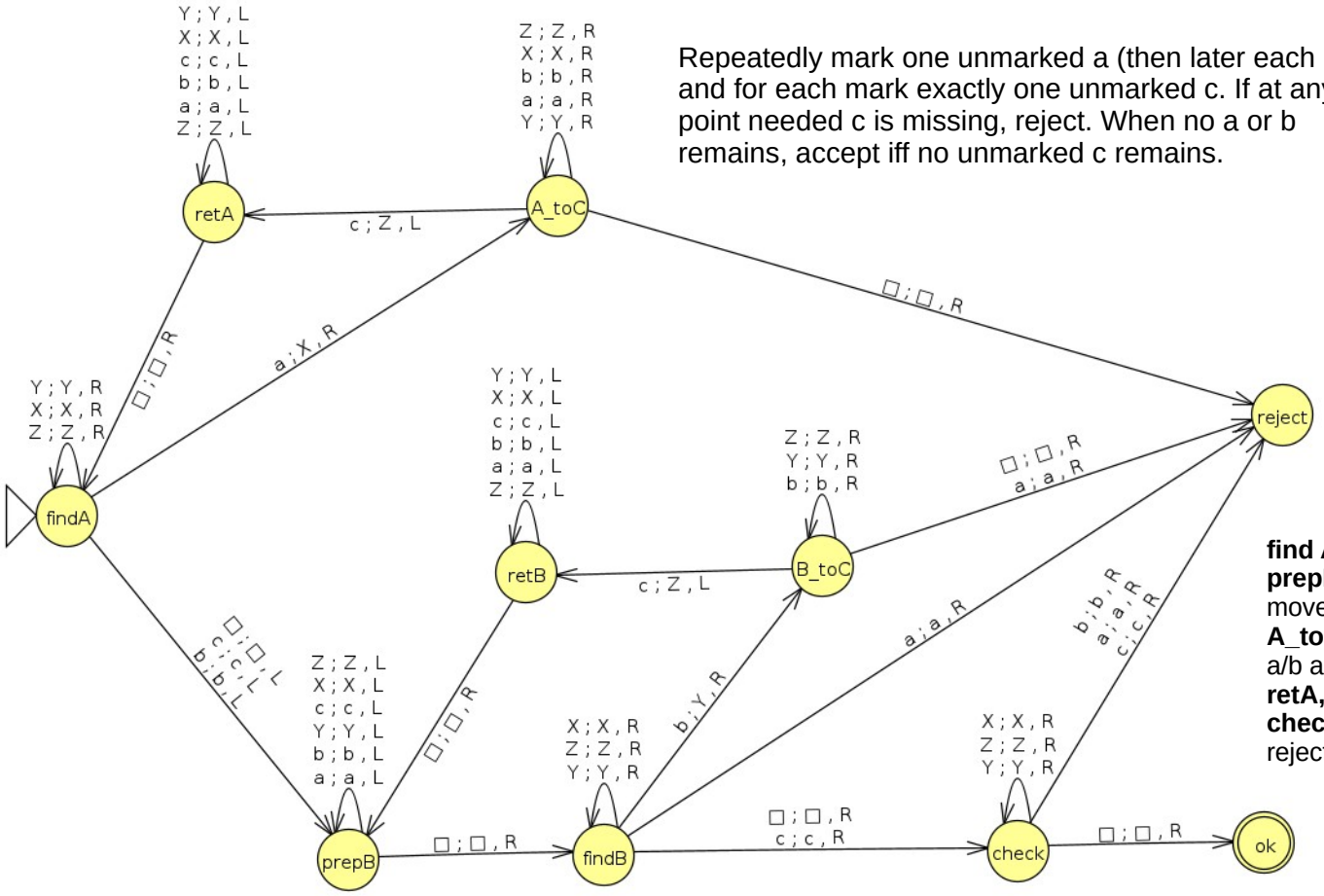
Strings that begin with p zeros, then r ones, and finally p−r zeros. The condition p ≥ r guarantees that the last block is not negative in length. The length of the final block matches the relationship between the number of initial zeros and the number of ones. For example 000100 and 0010. They can be recognized by PDA that pushes the initial zeros onto the stack, pops a zero for each 1, and then pops a zero for each of the final zeros.

X5 T5. In the task 1 of the previous exercises there was PDA.
The words accepted by the language are defined as $\{a^k b^l c^{k+l} \mid k,l \in N\}$.
For example aabbcccc. Construct the corresponding Turing machine.



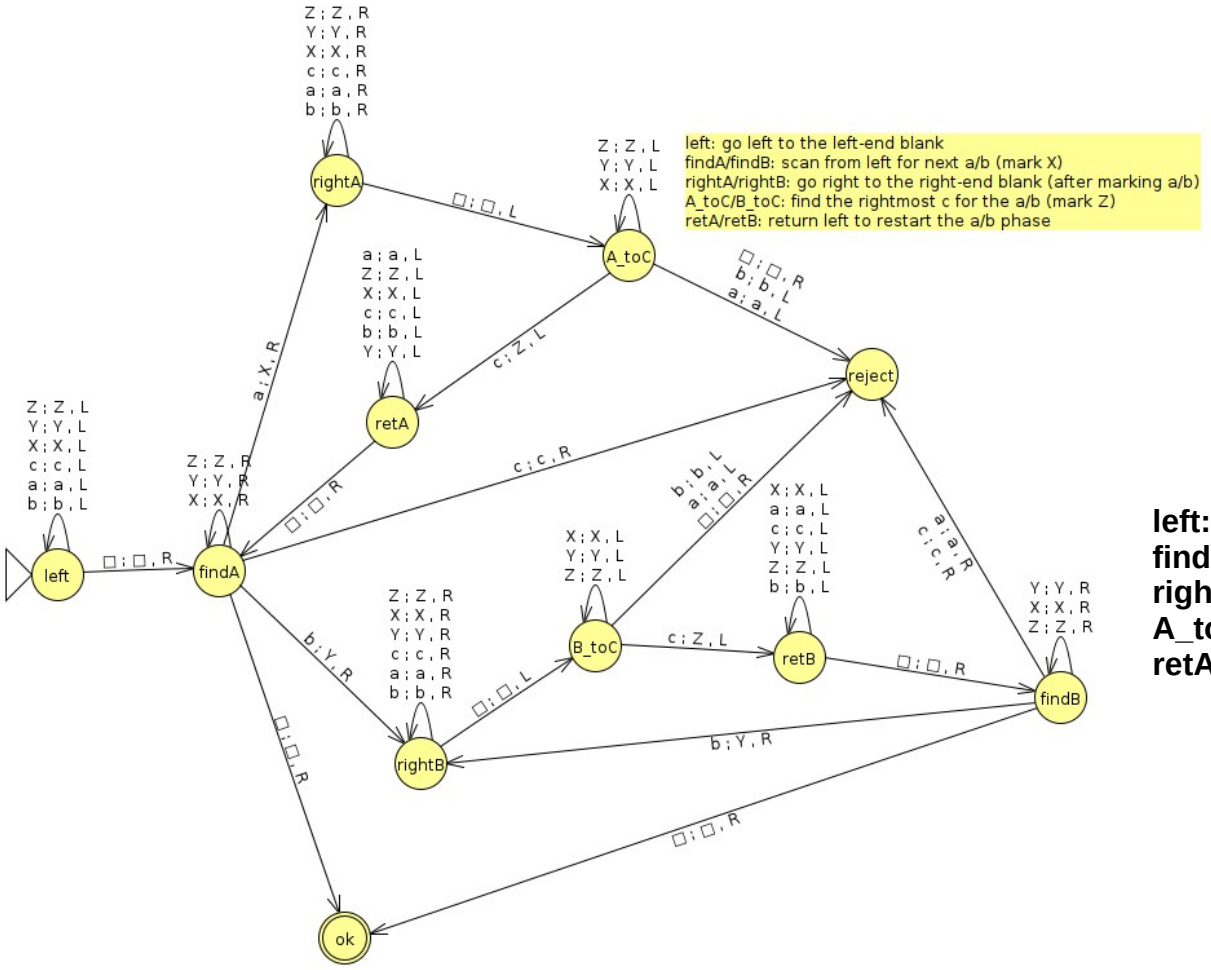
Repeatedly mark one unmarked a (then later each b) and for each mark exactly one unmarked c. If at any point needed c is missing, reject. When no a or b remains, accept iff no unmarked c remains.

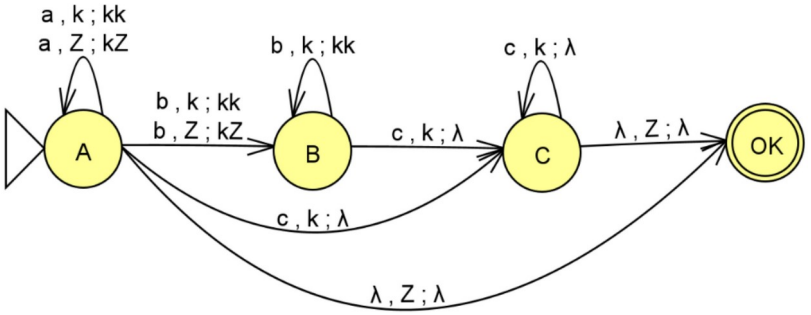
S->A
A->aAc
A->B
B->bBc
B-> ε



find A,findB - scan for an unmarked a/b, mark it and go find c.
prepB If no a is found, switch to the b phase.
move left to the tape start before scanning for b's.
A_toC/B_toC - move right to find unmarked c to match the last a/b and mark it. If none, reject.
retA,retB - move left to the tape start and go back to initial state
check - ensure there are no unmarked c's left. If any found, reject, otherwise go to OK.

X5 T5. In the task 1 of the previous exercises there was PDA.
The words accepted by the language are defined as $\{a^k b^l c^{k+l} \mid k,l \in N\}$.
For example aabbcccc. Construct the corresponding Turing machine.

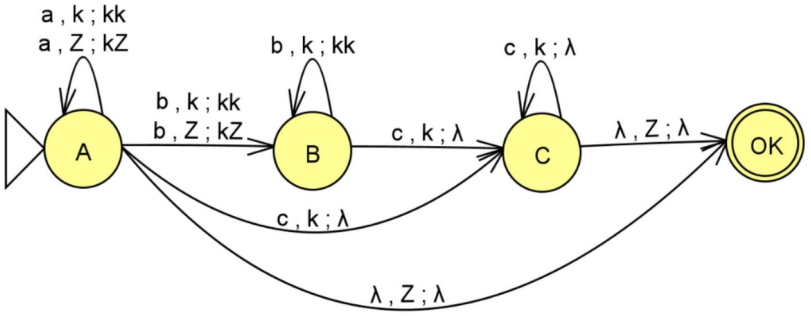


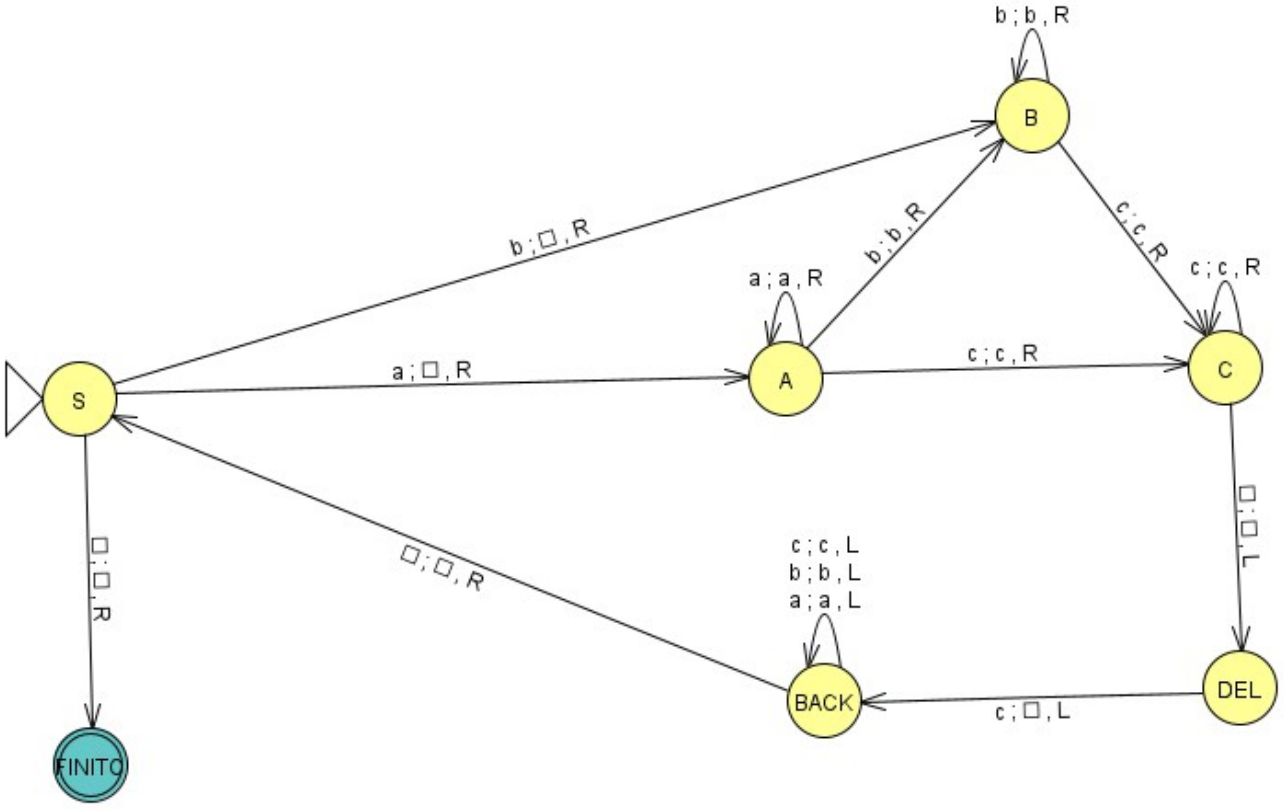


S->A
A->aAc
A->B
B->bBc
B-> ε

left: go left to the left-end blank
findA/findB: scan from left for next a/b (mark X)
rightA/rightB: go right to right-end blank (after marking a/b)
A_toC/B_toC: find the rightmost c for the a/b (mark Z)
retA/retB: return left to restart the a/b phase

X5 T5. In the task 1 of the previous exercises there was PDA.
The words accepted by the language are defined as $\{a^k b^l c^{k+l} \mid k,l \in N\}$.
For example aabbcccc. Construct the corresponding Turing machine.



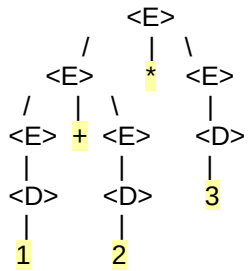


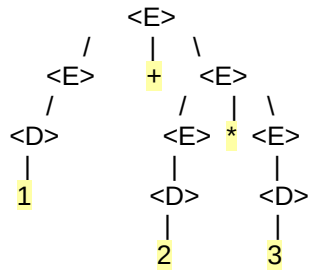
S->A
A->aAc
A->B
B->bBc
B-> ε

X5 T6. The following grammar was presented in the lectures. It was noted to be ambiguous and did not include operator precedence in any way. How would you modify the grammar to make it unambiguous?

<E> → <D>
<E> → (<E>)
<E> → <E> + <E>
<E> → <E> - <E>
<E> → <E> * <E>
<E> → <E> / <E>
<D> → 0 | 1 | 2 | ... 9

In the original grammar, all operators appear at the same level. The grammar does not enforce precedence: addition and multiplication are treated equally. For an input like 1+2*3, the grammar allows multiple parse trees, because it can either group 1+2 or 2*3. Both trees are valid according to the rules, but they give different evaluations.





<E> → <T> | <E> + <T> | <E> - <T>
<T> → <F> | <T> * <F> | <T> / <F>
<F> → <D> | (<E>)
<D> → 0 | 1 | 2 | ... | 9

The modified grammar follows normal math precedence.
<E> handles + and − because they have the lowest precedence, so expressions are split around them first.
<T> handles * and / because they come next and must be grouped inside each <E>.
<F> handles digits and parentheses, which have the highest precedence: parentheses force grouping, and digits are the basic units.

